```
import { useState, useEffect, useRef } from "react";

const TAU = Math.PI * 2;

// ── palette ──────────────────────────────────────────────────────────
const C = {
  obsidian: "#0a0b0d",
  titanium: "#1a1c20",
  chrome: "#8a9aaa",
  chromeDim: "#3a4550",
  pearl: "#d4e8f0",
  pearlDim: "#7ba8bc",
  accent: "#4dd9ff",
  accentGlow: "#00aaff",
  gold: "#c8a84b",
  goldDim: "#6a5820",
  red: "#ff4444",
  green: "#44ffaa",
  white: "#f0f4f8",
  fog: "rgba(77,217,255,0.06)",
};

// ── utility: polar → cartesian ──────────────────────────────────────
const polar = (cx, cy, r, a) => [
  cx + r * Math.cos(a - Math.PI / 2),
  cy + r * Math.sin(a - Math.PI / 2),
];

// ── data node seeds ──────────────────────────────────────────────────
function buildNodes(count, cx, cy, rMin, rMax, seed = 1) {
  const nodes = [];
  let s = seed;
  const rng = () => { s = (s * 1664525 + 1013904223) & 0xffffffff; return (s >>>
  for (let i = 0; i < count; i++) {
    const angle = rng() * TAU;
    const r = rMin + rng() * (rMax - rMin);
    const [x, y] = polar(cx, cy, r, angle + Math.PI / 2);
    const type = rng() < 0.15 ? "diamond" : rng() < 0.5 ? "square" : "dot";
    nodes.push({ x, y, r: 1 + rng() * 2.5, type, phase: rng() * TAU, speed: 0.5 +
  }
  return nodes;
}

// ── static annotation lines ──────────────────────────────────────────
```

```
const ANNOTATIONS = [
  { x1: 200, y1: 220, x2: 130, y2: 160, label: "BIOMETRIC SCAN · ACTIVE", anchor:
  { x1: 600, y1: 220, x2: 670, y2: 160, label: "NEURAL SIGNATURE · 97.4%", anchor
  { x1: 180, y1: 400, x2: 90,  y2: 400, label: "KINEMATIC PROFILE · CLASS-A", anc
  { x1: 620, y1: 400, x2: 710, y2: 400, label: "THREAT VECTOR · NULL", anchor: "s
  { x1: 200, y1: 560, x2: 130, y2: 620, label: "METABOLIC INDEX · 0.94", anchor:
  { x1: 600, y1: 560, x2: 670, y2: 620, label: "TEMPORAL DRIFT · +0.002ms", ancho
  { x1: 400, y1: 100, x2: 400, y2:  40, label: "CAPTURE MODE · NEUTRAL", anchor:
  { x1: 400, y1: 700, x2: 400, y2: 760, label: "CONTAINMENT STATUS · NOMINAL", an
];

const RING_DATA = [
  { r: 220, dash: "4 3", speed: 0.3, rev: false, label: "RING-01 · BIOMETRIC", op
  { r: 235, dash: "1 6", speed: 0.15, rev: true, label: "RING-02 · CHROMATIC", op
  { r: 250, dash: "8 2", speed: 0.08, rev: false, label: "RING-03 · KINEMATIC", o
  { r: 265, dash: "2 5", speed: 0.2,  rev: true,  label: "RING-04 · NEURAL", opac
  { r: 285, dash: "6 4", speed: 0.05, rev: false, label: "RING-05 · TEMPORAL", op
  { r: 310, dash: "3 3", speed: 0.12, rev: true,  label: "RING-06 · QUANTUM", opa
  { r: 340, dash: "10 3",speed: 0.04, rev: false, label: "RING-07 · SPECTRAL", op
];

// ── tick marks per ring ──────────────────────────────────────────────
function RingTicks({ cx, cy, r, count = 60, color, opacity = 0.5 }) {
  return Array.from({ length: count }, (_, i) => {
    const a = (i / count) * TAU;
    const len = i % 5 === 0 ? 8 : 3;
    const [x1, y1] = polar(cx, cy, r, a + Math.PI / 2);
    const [x2, y2] = polar(cx, cy, r + len, a + Math.PI / 2);
    return <line key={i} x1={x1} y1={y1} x2={x2} y2={y2} stroke={color} strokeWid
  });
}

// ── inset panel ──────────────────────────────────────────────────────
function InsetPanel({ x, y, w, h, title, children }) {
  return (
    <g transform={`translate(${x},${y})`}>
      <rect width={w} height={h} rx={3} fill="rgba(10,11,13,0.85)" stroke={C.chro
      <rect width={w} height={12} rx={3} fill={C.titanium} stroke="none" />
      <text x={4} y={9} fontSize={6} fill={C.pearlDim} fontFamily="'Courier New',
      {children}
    </g>
  );
}

// ── sparkline ────────────────────────────────────────────────────────
function Sparkline({ x, y, w, h, color, seed = 7 }) {
  let s = seed;
```

```
  const rng = () => { s = (s * 1664525 + 1013904223) & 0xffffffff; return (s >>>
  const pts = Array.from({ length: 20 }, (_, i) => {
    const px = x + (i / 19) * w;
    const py = y + h - rng() * h;
    return `${px},${py}`;
  }).join(" ");
  return <polyline points={pts} fill="none" stroke={color} strokeWidth={0.8} opac
}

// ── bar graph ──────────────────────────────────────────────────────────
function BarGraph({ x, y, w, h, color, seed = 3 }) {
  let s = seed;
  const rng = () => { s = (s * 1664525 + 1013904223) & 0xffffffff; return (s >>>
  const bars = Array.from({ length: 10 }, (_, i) => {
    const bh = rng() * h;
    return <rect key={i} x={x + i * (w / 10)} y={y + h - bh} width={w / 10 - 1} h
  });
  return <>{bars}</>;
}

// ── main component ─────────────────────────────────────────────────────
export default function FGEMagicMirror() {
  const [tick, setTick] = useState(0);
  const [scanLine, setScanLine] = useState(0);
  const [boot, setBoot] = useState(false);
  const [glitch, setGlitch] = useState(false);
  const reqRef = useRef();
  const startRef = useRef(null);

  useEffect(() => {
    setTimeout(() => setBoot(true), 400);
    const glitchInterval = setInterval(() => {
      setGlitch(true);
      setTimeout(() => setGlitch(false), 80);
    }, 4000);
    const animate = (ts) => {
      if (!startRef.current) startRef.current = ts;
      const t = (ts - startRef.current) / 1000;
      setTick(t);
      setScanLine((t * 60) % 400);
      reqRef.current = requestAnimationFrame(animate);
    };
    reqRef.current = requestAnimationFrame(animate);
    return () => { cancelAnimationFrame(reqRef.current); clearInterval(glitchInte
  }, []);

  const CX = 400, CY = 400;
```

```
const nodes = buildNodes(1000, CX, CY, 360, 520, 42);

return (
  <div style={{
    background: "#050608",
    minHeight: "100vh",
    display: "flex",
    alignItems: "center",
    justifyContent: "center",
    fontFamily: "'Courier New', monospace",
    overflow: "hidden",
    position: "relative",
  }}>
    {/* ambient vignette */}
    <div style={{
      position: "absolute", inset: 0,
      background: "radial-gradient(ellipse 70% 70% at 50% 50%, transparent 40%,
      pointerEvents: "none", zIndex: 10,
    }} />

    {/* corner FGE insignia */}
    {["TL","TR","BL","BR"].map(corner => (
      <div key={corner} style={{
        position: "absolute",
        top: corner.startsWith("T") ? 24 : "auto",
        bottom: corner.startsWith("B") ? 24 : "auto",
        left: corner.endsWith("L") ? 24 : "auto",
        right: corner.endsWith("R") ? 24 : "auto",
        fontSize: 9, letterSpacing: 3, color: C.chromeDim,
        opacity: boot ? 1 : 0, transition: "opacity 1.2s",
      }}>
        FGE · {corner === "TL" ? "VAULT-7" : corner === "TR" ? "CLASSIFIED" : c
      </div>
    ))}

    {/* top status bar */}
    <div style={{
      position: "absolute", top: 12, left: "50%", transform: "translateX(-50%)"
      fontSize: 8, letterSpacing: 4, color: C.accent, opacity: boot ? 0.8 : 0,
      transition: "opacity 1s", textAlign: "center",
    }}>
      FERAL GLOSS EMPIRE · CANON ENGINE v2.1 · MAGIC MIRROR PROTOCOL · TOP SECR
    </div>

    {/* bottom status bar */}
    <div style={{
      position: "absolute", bottom: 12, left: "50%", transform: "translateX(-50
```

```
  fontSize: 7, letterSpacing: 3, color: C.chromeDim, opacity: boot ? 0.6 :
  transition: "opacity 1.4s",
}}>
  SAFE LOOP · SIGNAL TOPOLOGY MAP · CYCLE SHAPE SIGNATURE · IREEEI LEARNING
</div>

{/* main SVG canvas */}
<svg
  width={800} height={800}
  viewBox="0 0 800 800"
  style={{
    filter: glitch ? "hue-rotate(20deg) saturate(2)" : "none",
    transition: glitch ? "none" : "filter 0.1s",
    opacity: boot ? 1 : 0,
    transform: boot ? "scale(1)" : "scale(0.94)",
    transition: `opacity 1.2s, transform 1.2s`,
  }}
>
  <defs>
    {/* radial background glow */}
    <radialGradient id="bgGlow" cx="50%" cy="50%" r="50%">
      <stop offset="0%" stopColor="#0d1a24" />
      <stop offset="60%" stopColor="#070a0d" />
      <stop offset="100%" stopColor="#020304" />
    </radialGradient>

    {/* mirror surface gradient */}
    <radialGradient id="mirrorGrad" cx="45%" cy="42%" r="55%">
      <stop offset="0%" stopColor="#1a2530" />
      <stop offset="35%" stopColor="#0d1318" />
      <stop offset="70%" stopColor="#070c10" />
      <stop offset="100%" stopColor="#030507" />
    </radialGradient>

    {/* mirror rim gradient */}
    <linearGradient id="rimGrad" x1="0%" y1="0%" x2="100%" y2="100%">
      <stop offset="0%" stopColor="#6a7a8a" />
      <stop offset="30%" stopColor="#c8d4dc" />
      <stop offset="50%" stopColor="#e8f0f4" />
      <stop offset="70%" stopColor="#8090a0" />
      <stop offset="100%" stopColor="#303840" />
    </linearGradient>

    {/* outer ring glow */}
    <radialGradient id="outerGlow" cx="50%" cy="50%" r="50%">
      <stop offset="75%" stopColor="transparent" />
      <stop offset="88%" stopColor="rgba(77,217,255,0.04)" />
```

```
    <stop offset="95%" stopColor="rgba(77,217,255,0.08)" />
    <stop offset="100%" stopColor="transparent" />
  </radialGradient>

  {/* scan line gradient */}
  <linearGradient id="scanGrad" x1="0%" y1="0%" x2="0%" y2="100%">
    <stop offset="0%" stopColor="transparent" />
    <stop offset="50%" stopColor="rgba(77,217,255,0.12)" />
    <stop offset="100%" stopColor="transparent" />
  </linearGradient>

  {/* clip for mirror interior */}
  <clipPath id="mirrorClip">
    <circle cx={CX} cy={CY} r={198} />
  </clipPath>

  {/* glow filter */}
  <filter id="glow" x="-50%" y="-50%" width="200%" height="200%">
    <feGaussianBlur stdDeviation="3" result="blur" />
    <feMerge><feMergeNode in="blur" /><feMergeNode in="SourceGraphic" /><
  </filter>
  <filter id="glowStrong" x="-100%" y="-100%" width="300%" height="300%">
    <feGaussianBlur stdDeviation="8" result="blur" />
    <feMerge><feMergeNode in="blur" /><feMergeNode in="SourceGraphic" /><
  </filter>
  <filter id="softBlur">
    <feGaussianBlur stdDeviation="1.5" />
  </filter>

  {/* underglow */}
  <radialGradient id="underGlow" cx="50%" cy="85%" r="40%">
    <stop offset="0%" stopColor="rgba(77,217,255,0.15)" />
    <stop offset="100%" stopColor="transparent" />
  </radialGradient>
</defs>

{/* background */}
<rect width={800} height={800} fill="url(#bgGlow)" />
<rect width={800} height={800} fill="url(#underGlow)" />

{/* outer ambient glow ring */}
<circle cx={CX} cy={CY} r={380} fill="url(#outerGlow)" />

{/* ── volumetric haze rings ── */}
{[360, 350, 338].map((r, i) => (
  <circle key={i} cx={CX} cy={CY} r={r}
    fill="none" stroke={C.accentGlow}
```

```
    strokeWidth={i === 0 ? 0.3 : 0.2}
    opacity={0.04 + i * 0.01}
    filter="url(#softBlur)"
  />
))}

{/* ── 1000 data nodes ── */}
{nodes.map((n, i) => {
  const pulse = Math.sin(tick * n.speed + n.phase);
  const vis = 0.3 + pulse * 0.35;
  return n.type === "diamond" ? (
    <rect key={i}
      x={n.x - n.r} y={n.y - n.r}
      width={n.r * 2} height={n.r * 2}
      fill={i % 7 === 0 ? C.gold : C.accent}
      opacity={vis}
      transform={`rotate(45,${n.x},${n.y})`}
    />
  ) : n.type === "square" ? (
    <rect key={i}
      x={n.x - n.r * 0.8} y={n.y - n.r * 0.8}
      width={n.r * 1.6} height={n.r * 1.6}
      fill={i % 5 === 0 ? C.pearlDim : C.chromeDim}
      opacity={vis * 0.8}
    />
  ) : (
    <circle key={i} cx={n.x} cy={n.y} r={n.r}
      fill={i % 11 === 0 ? C.green : i % 13 === 0 ? C.gold : C.accent}
      opacity={vis * 0.7}
    />
  );
})}

{/* ── rotating diagnostic rings ── */}
{RING_DATA.map((ring, idx) => {
  const rot = (tick * ring.speed * (ring.rev ? -1 : 1) * 360) % 360;
  return (
    <g key={idx} transform={`rotate(${rot},${CX},${CY})`}>
      <circle cx={CX} cy={CY} r={ring.r}
        fill="none"
        stroke={idx % 2 === 0 ? C.accent : C.chrome}
        strokeWidth={0.6}
        strokeDasharray={ring.dash}
        opacity={ring.opacity}
      />
      <RingTicks cx={CX} cy={CY} r={ring.r - 6} count={idx % 2 === 0 ? 72
    </g>
```

```
  );
})}

{/* outer static precision ring */}
<circle cx={CX} cy={CY} r={345} fill="none" stroke={C.chromeDim} strokeWi
<circle cx={CX} cy={CY} r={348} fill="none" stroke={C.chrome} strokeWidth

{/* degree tick marks on outer ring */}
{Array.from({ length: 360 }, (_, i) => {
  const a = (i / 360) * TAU;
  const len = i % 90 === 0 ? 12 : i % 30 === 0 ? 8 : i % 10 === 0 ? 5 : 2
  const [x1, y1] = polar(CX, CY, 345, a + Math.PI / 2);
  const [x2, y2] = polar(CX, CY, 345 + len, a + Math.PI / 2);
  return <line key={i} x1={x1} y1={y1} x2={x2} y2={y2}
    stroke={C.chrome} strokeWidth={i % 30 === 0 ? 0.8 : 0.3}
    opacity={i % 30 === 0 ? 0.6 : 0.2} />;
})}

{/* cardinal degree labels */}
{[0, 90, 180, 270].map(deg => {
  const a = (deg / 360) * TAU;
  const [x, y] = polar(CX, CY, 365, a + Math.PI / 2);
  return <text key={deg} x={x} y={y + 3} textAnchor="middle"
    fontSize={7} fill={C.pearlDim} opacity={0.7}
    fontFamily="'Courier New',monospace">{deg}°</text>;
})}

{/* ── mirror body: 3D layered rings for depth ── */}
{[210, 207, 204, 201].map((r, i) => (
  <circle key={i} cx={CX} cy={CY} r={r}
    fill="none"
    stroke={`rgba(${i === 0 ? "200,212,220" : i === 1 ? "130,154,170" : i
    strokeWidth={i === 0 ? 2.5 : i === 1 ? 1.5 : i === 2 ? 1 : 0.5}
  />
))}

{/* rim highlight (satin titanium) */}
<circle cx={CX} cy={CY} r={204}
  fill="none"
  stroke="url(#rimGrad)"
  strokeWidth={4}
  opacity={0.9}
/>

{/* mirror interior fill */}
<circle cx={CX} cy={CY} r={199} fill="url(#mirrorGrad)" />
```

```
{/* ── scan line sweep ── */}
<g clipPath="url(#mirrorClip)">
  <rect x={CX - 199} y={CY - 199 + scanLine - 20} width={398} height={40}
    fill="url(#scanGrad)" opacity={0.6} />
  <line x1={CX - 199} y1={CY - 199 + scanLine}
        x2={CX + 199} y2={CY - 199 + scanLine}
    stroke={C.accent} strokeWidth={0.4} opacity={0.4} />
</g>

{/* ── mirror surface specular highlights ── */}
<ellipse cx={CX - 55} cy={CY - 70} rx={40} ry={20}
  fill="rgba(212,232,240,0.06)" transform={`rotate(-20,${CX - 55},${CY -
<ellipse cx={CX + 30} cy={CY - 90} rx={18} ry={8}
  fill="rgba(212,232,240,0.04)" transform={`rotate(15,${CX + 30},${CY - 9

{/* ── pearlescent rim light arc ── */}
<path
  d={`M ${CX - 185},${CY - 70} A 199 199 0 0 1 ${CX + 100},${CY - 175}`}
  fill="none" stroke="rgba(212,232,240,0.35)" strokeWidth={2}
  filter="url(#softBlur)"
/>

{/* ── subject placeholder (120mm capture zone) ── */}
<g clipPath="url(#mirrorClip)">
  {/* subject silhouette frame */}
  <rect x={CX - 55} y={CY - 140} width={110} height={220}
    fill="rgba(77,217,255,0.03)" stroke={C.accent}
    strokeWidth={0.5} strokeDasharray="3 3" opacity={0.5} rx={2} />

  {/* corner brackets */}
  {[[-55,-140],[45,-140],[-55,80],[45,80]].map(([dx,dy],i) => {
    const xSign = dx < 0 ? 1 : -1;
    const ySign = dy < 0 ? 1 : -1;
    return (
      <g key={i}>
        <line x1={CX+dx} y1={CY+dy} x2={CX+dx+xSign*12} y2={CY+dy} stroke
        <line x1={CX+dx} y1={CY+dy} x2={CX+dx} y2={CY+dy+ySign*12} stroke
      </g>
    );
  })}

  {/* crosshair */}
  <line x1={CX - 10} y1={CY - 30} x2={CX + 10} y2={CY - 30} stroke={C.acc
  <line x1={CX} y1={CY - 40} x2={CX} y2={CY - 20} stroke={C.accent} strok

  {/* center reticle */}
  <circle cx={CX} cy={CY - 30} r={3} fill="none" stroke={C.accent} stroke
```

```
  {/* subject label */}
  <text x={CX} y={CY + 100} textAnchor="middle" fontSize={6}
    fill={C.pearlDim} fontFamily="'Courier New',monospace"
    letterSpacing={2} opacity={0.7}>SUBJECT · NEUTRAL CAPTURE</text>
</g>

{/* ── inner capture ring ── */}
<circle cx={CX} cy={CY - 30} r={120} fill="none"
  stroke={C.accent} strokeWidth={0.4}
  strokeDasharray="2 4" opacity={0.3}
  transform={`rotate(${tick * 8},${CX},${CY - 30})`} />

{/* ── FGE central emblem ── */}
<g filter="url(#glowStrong)" opacity={0.9}>
  <text x={CX} y={CY + 160} textAnchor="middle"
    fontSize={11} fill={C.gold}
    fontFamily="'Courier New',monospace"
    letterSpacing={6}>⬡ FGE ⬡</text>
</g>
<text x={CX} y={CY + 174} textAnchor="middle"
  fontSize={5.5} fill={C.chromeDim}
  fontFamily="'Courier New',monospace"
  letterSpacing={3}>FERAL GLOSS EMPIRE</text>

{/* ── annotation lines ── */}
{ANNOTATIONS.map((ann, i) => {
  const mx = ann.anchor === "end" ? ann.x2 - 40 : ann.anchor === "start"
  const my = ann.y2;
  return (
    <g key={i} opacity={0.75}>
      <line x1={ann.x1} y1={ann.y1} x2={mx} y2={my}
        stroke={C.chrome} strokeWidth={0.4} strokeDasharray="2 3" />
      <line x1={mx} y1={my} x2={ann.x2} y2={ann.y2}
        stroke={C.chrome} strokeWidth={0.4} />
      <circle cx={ann.x1} cy={ann.y1} r={1.5} fill={C.accent} opacity={0.
      <text x={ann.x2 + (ann.anchor === "end" ? -4 : ann.anchor === "star
        y={ann.y2 - 3}
        textAnchor={ann.anchor}
        fontSize={5.5} fill={C.white}
        fontFamily="'Courier New',monospace"
        letterSpacing={1.5} opacity={0.85}>{ann.label}</text>
    </g>
  );
})}

{/* ── inset analysis panels ── */}
```

```
{/* top-left: waveform */}
<InsetPanel x={30} y={280} w={100} h={70} title="BIOMETRIC · WAVEFORM">
  <Sparkline x={4} y={16} w={92} h={46} color={C.accent} seed={11} />
  <text x={4} y={68} fontSize={5} fill={C.chromeDim} fontFamily="'Courier
</InsetPanel>

{/* top-right: bar spectrum */}
<InsetPanel x={670} y={280} w={100} h={70} title="SPECTRAL · DENSITY">
  <BarGraph x={4} y={15} w={92} h={46} color={C.accentGlow} seed={23} />
  <text x={4} y={68} fontSize={5} fill={C.chromeDim} fontFamily="'Courier
</InsetPanel>

{/* bottom-left: neural map */}
<InsetPanel x={30} y={440} w={100} h={70} title="NEURAL · SIGNATURE">
  <Sparkline x={4} y={16} w={92} h={46} color={C.green} seed={37} />
  <text x={4} y={68} fontSize={5} fill={C.chromeDim} fontFamily="'Courier
</InsetPanel>

{/* bottom-right: kinematic */}
<InsetPanel x={670} y={440} w={100} h={70} title="KINEMATIC · PROFILE">
  <BarGraph x={4} y={15} w={92} h={46} color={C.gold} seed={59} />
  <text x={4} y={68} fontSize={5} fill={C.chromeDim} fontFamily="'Courier
</InsetPanel>

{/* top center mini panel */}
<InsetPanel x={310} y={26} w={180} h={40} title="CAPTURE · DIAGNOSTICS">
  <text x={4} y={24} fontSize={5.5} fill={C.accent} fontFamily="'Courier
  <text x={4} y={34} fontSize={5.5} fill={C.chromeDim} fontFamily="'Couri
</InsetPanel>

{/* bottom center mini panel */}
<InsetPanel x={310} y={734} w={180} h={40} title="CONTAINMENT · STATUS">
  <text x={4} y={24} fontSize={5.5} fill={C.green} fontFamily="'Courier N
  <text x={4} y={34} fontSize={5.5} fill={C.chromeDim} fontFamily="'Couri
</InsetPanel>

{/* ── radial quadrant labels ── */}
{[
  { a: 315, label: "SECTOR·A" },
  { a: 45,  label: "SECTOR·B" },
  { a: 135, label: "SECTOR·C" },
  { a: 225, label: "SECTOR·D" },
].map(({ a, label }) => {
  const [x, y] = polar(CX, CY, 328, (a / 360) * TAU + Math.PI / 2);
  return <text key={a} x={x} y={y + 3} textAnchor="middle"
    fontSize={5.5} fill={C.goldDim}
```

```
    fontFamily="'Courier New',monospace"
    letterSpacing={2} opacity={0.6}>{label}</text>;
})}

{/* ── radial spoke lines ── */}
{[0, 45, 90, 135, 180, 225, 270, 315].map((deg, i) => {
  const a = (deg / 360) * TAU;
  const [x1, y1] = polar(CX, CY, 215, a + Math.PI / 2);
  const [x2, y2] = polar(CX, CY, 290, a + Math.PI / 2);
  return <line key={i} x1={x1} y1={y1} x2={x2} y2={y2}
    stroke={C.chromeDim} strokeWidth={0.3}
    opacity={0.3} />;
})}

{/* ── pulsing accent dots on rim ── */}
{[0, 60, 120, 180, 240, 300].map((deg, i) => {
  const a = (deg / 360) * TAU;
  const [x, y] = polar(CX, CY, 204, a + Math.PI / 2);
  const pulse = 0.5 + 0.5 * Math.sin(tick * 2 + i);
  return (
    <g key={i}>
      <circle cx={x} cy={y} r={3} fill={C.accent} opacity={pulse * 0.9} f
      <circle cx={x} cy={y} r={1.5} fill={C.white} opacity={pulse} />
    </g>
  );
})}

{/* ── live readouts near rim ── */}
{[
  { a: 0,   val: () => (98.2 + Math.sin(tick * 0.7) * 0.5).toFixed(1) + "
  { a: 60,  val: () => (1.44 + Math.sin(tick * 1.1) * 0.02).toFixed(2) +
  { a: 120, val: () => (0.97 + Math.sin(tick * 0.9) * 0.01).toFixed(2) },
  { a: 180, val: () => "0.00%" },
  { a: 240, val: () => (2 + Math.round(Math.sin(tick * 2) * 0.5)) + "ms"
  { a: 300, val: () => (64 + Math.round(Math.sin(tick * 0.4) * 2)) + "" }
].map(({ a, val }, i) => {
  const [x, y] = polar(CX, CY, 245, (a / 360) * TAU + Math.PI / 2);
  return <text key={i} x={x} y={y + 3}
    textAnchor="middle" fontSize={6}
    fill={C.pearlDim} fontFamily="'Courier New',monospace"
    opacity={0.65}>{val()}</text>;
})}

{/* ── top header decoration ── */}
<line x1={60} y1={20} x2={300} y2={20} stroke={C.chromeDim} strokeWidth={
<line x1={500} y1={20} x2={740} y2={20} stroke={C.chromeDim} strokeWidth=
<line x1={60} y1={780} x2={300} y2={780} stroke={C.chromeDim} strokeWidth
```

```
        <line x1={500} y1={780} x2={740} y2={780} stroke={C.chromeDim} strokeWidt

        {/* ── classification band ── */}
        <rect x={0} y={0} width={800} height={6} fill={C.red} opacity={0.7} />
        <rect x={0} y={794} width={800} height={6} fill={C.red} opacity={0.7} />
        <text x={400} y={4.5} textAnchor="middle" fontSize={4} fill={C.white}
          fontFamily="'Courier New',monospace" letterSpacing={6}>
          ⚠ TOP SECRET · COMPARTMENTALIZED · FGE INTERNAL USE ONLY ⚠
        </text>
        <text x={400} y={799} textAnchor="middle" fontSize={4} fill={C.white}
          fontFamily="'Courier New',monospace" letterSpacing={6}>
          ⚠ UNAUTHORIZED ACCESS PROSECUTED · CANON ENGINE v2.1 · CYCLE: {Math.flo
        </text>

        {/* ── underglow atmospheric streak ── */}
        <ellipse cx={CX} cy={CY + 220} rx={180} ry={20}
          fill="rgba(77,217,255,0.07)" filter="url(#softBlur)" />
      </svg>
    </div>
  );
}
```